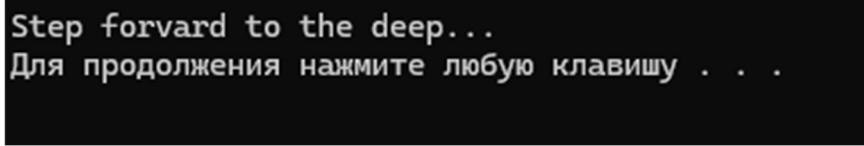


Разминка. Этап 1

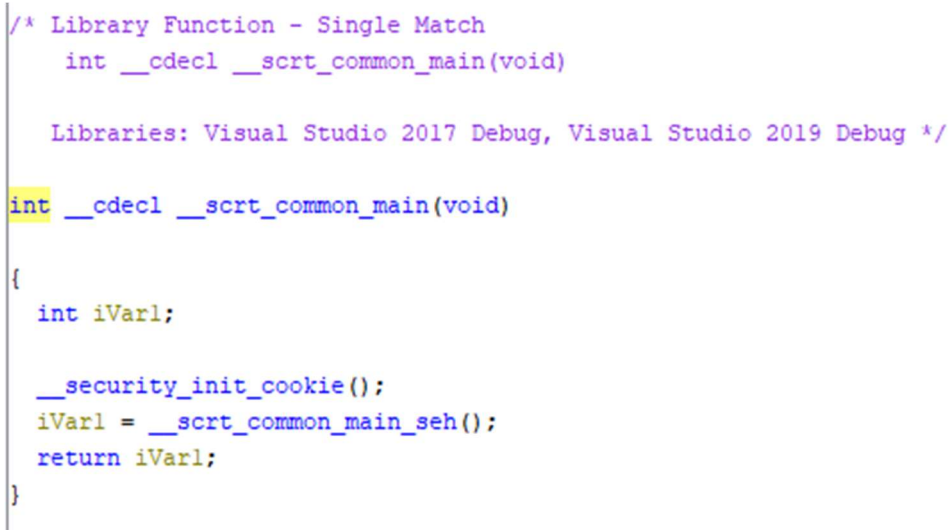
При открытии программы видим, что она не выводит ничего конкретного, только текстовую строку



```
Step forward to the deep...
Для продолжения нажмите любую клавишу . . .
```

Так как task рассчитан на совсем новичков, собственно почему разминка и называется, то это больше хинт куда смотреть. Надо смотреть глубже и поэтому идем в дизассемблер (гидру)

Открываем в дизассемблере. Анализ точки входа ничего не показал



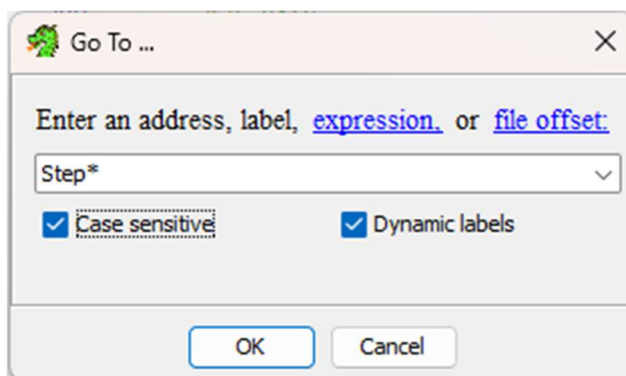
```
/* Library Function - Single Match
   int __cdecl __scrt_common_main(void)

   Libraries: Visual Studio 2017 Debug, Visual Studio 2019 Debug */

int __cdecl __scrt_common_main(void)
{
    int iVar1;

    __security_init_cookie();
    iVar1 = __scrt_common_main_seh();
    return iVar1;
}
```

Вспоминаем вывод в консоль, ищем вхождение строки (или её части)



Переходим, видим следующее

```

s_string_too_long_14001e280                                XREF[1]:  FUN_140014890:1400148ac(*)
14001e280 73 74 72      ds      "string too long"
          69 6e 67
          20 74 6f ...

14001e290 00          ??      00h
14001e291 00          ??      00h
14001e292 00          ??      00h
14001e293 00          ??      00h
14001e294 00          ??      00h
14001e295 00          ??      00h
14001e296 00          ??      00h
14001e297 00          ??      00h

s_FECTF(first_steps)_14001e298                            XREF[1]:  FUN_140014f40:140014f7e(*)
14001e298 46 45 43      ds      "FECTF{first_steps}"
          54 46 7b
          66 69 72 ...

14001e2ab 00          ??      00h
14001e2ac 00          ??      00h
14001e2ad 00          ??      00h
14001e2ae 00          ??      00h
14001e2af 00          ??      00h

s_Step_forward_to_the_deep..._14001e2b0                  XREF[1]:  FUN_140014f40:140014f8f(*)
14001e2b0 53 74 65      ds      "Step forward to the deep..."
          70 20 66
          6f 72 76 ...

14001e2cc 00          ??      00h
14001e2cd 00          ??      00h
14001e2ce 00          ??      00h
14001e2cf 00          ??      00h
14001e2d0 00          ??      00h
14001e2d1 00          ??      00h
14001e2d2 00          ??      00h
14001e2d3 00          ??      00h

s_pause_14001e2d4                                         XREF[1]:  FUN_140014f40:140014fb3(*)
14001e2d4 70 61 75      ds      "pause"
          73 65 00

14001e2da 00          ??      00h
14001e2db 00          ??      00h
14001e2dc 00          ??      00h
14001e2dd 00          ??      00h
14001e2de 00          ??      00h
14001e2df 00          ??      00h

```

В принципе, флаг можно увидеть уже здесь – FECTF{first_steps}, но можно пойти чуть дальше. Видим референс на функцию, ждем туда. Видим дизассемблированную функцию, в которой без какой-либо обфускации и шифрования лежит флаг

FECTF{first_steps}

```

void FUN_140014f40(void)
{
    basic_ostream<> *this;
    longlong lVar1;
    undefined4 *puVar2;
    undefined1 local_158 [32];
    undefined4 local_138 [2];
    longlong local_130 [31];
    undefined4 local_34;
    ulonglong local_20;

    puVar2 = local_138;
    for (lVar1 = 0x1a; lVar1 != 0; lVar1 = lVar1 + -1) {
        *puVar2 = 0xffffffff;
        puVar2 = puVar2 + 1;
    }
    local_20 = DAT_140022040 ^ (ulonglong)local_138;
    __CheckForDebuggerJustMyCode(&DAT_14002806a);
    thunk_FUN_140013140(local_130, "FECTF{first_steps}");
    this = (basic_ostream<> *)
        thunk_FUN_140011d70((longlong *)cout_exref, "Step forward to the deep...");
    std::basic_ostream<>::operator<<
        (this, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011046);
    system("pause");
    local_34 = 0;
    thunk_FUN_140013960(local_130);
    _RTC_CheckStackVars((longlong)local_158, (int *)&DAT_14001de00);
    thunk_FUN_140015700(local_20 ^ (ulonglong)local_138);
    return;
}

```

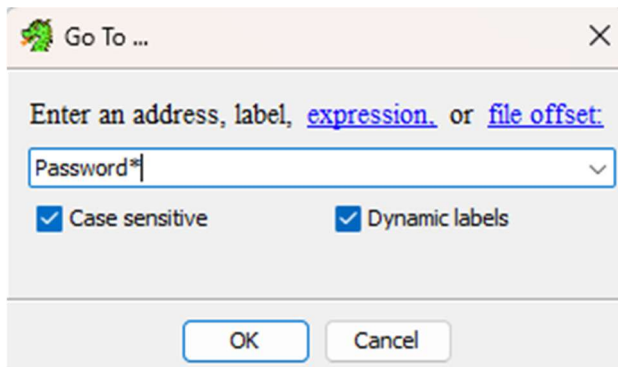
Разминка. Этап 2

Программа запрашивает пароль, и на первый введённый (конечно же) выдает, что пароль неверен.

Идём в гидру, открываем программу. Анализ точки входа снова ничего не показал.

```
void entry(void)
{
    __scrt_common_main();
    return;
}
```

Поэтому ищем вхождение строки запроса пароля



В целом, пароль тоже лежит выше, но мы перейдем к функции, где строка упоминается

```

void FUN_140019040(void)
{
    char cVar1;
    longlong lVar2;
    undefined4 *puVar3;
    undefined1 local_ld8 [32];
    undefined4 local_lb8 [2];
    longlong local_lb0 [8];
    longlong local_170 [8];
    longlong local_130 [31];
    undefined4 local_34;
    ulonglong local_20;

    puVar3 = local_lb8;
    for (lVar2 = 0x3a; lVar2 != 0; lVar2 = lVar2 + -1) {
        *puVar3 = 0xffffffff;
        puVar3 = puVar3 + 1;
    }
    local_20 = DAT_140029040 ^ (ulonglong)local_lb8;
    __CheckForDebuggerJustMyCode(&DAT_14002f06b);
    thunk_FUN_140015c10(local_lb0, "youshallnotpass");
    thunk_FUN_140015c10(local_170, "");
    thunk_FUN_1400139d0((longlong *)cout_exref, "Password:\t");
    thunk_FUN_140013510((longlong *)cin_exref, local_170);
    thunk_FUN_140018bd0(local_130);
    cVar1 = thunk_FUN_140013e70((longlong)local_170, (longlong)local_lb0);
    if (cVar1 == '\0') {
        thunk_FUN_1400139d0((longlong *)cout_exref, "Wrong, think again\n");
    }
    else {
        thunk_FUN_140013950((longlong *)cout_exref, (longlong)local_130);
    }
    system("pause");
    local_34 = 0;
    thunk_FUN_1400166e0(local_130);
    thunk_FUN_1400166e0(local_170);
    thunk_FUN_1400166e0(local_lb0);
    _RTC_CheckStackVars((longlong)local_ld8, (int *)&DAT_1400241c0);
    thunk_FUN_140019bf0(local_20 ^ (ulonglong)local_lb8);
    return;
}

```

Видим инструкцию cin_exref – функция ввода, записывает в переменную local_170

Обращаем внимание, что далее идёт функция сравнения по переменной cVar1

А сама переменная cVar1 изначально является результатом какой-то функции (в нашем случае – сравнения) двух переменных – нашей local_170 и local_lb0

В local_lb0 лежит строка youshallnotpass

Исходя из логики программы, предполагаем, что это и есть наш искомый пароль.
Введём его, получим флаг FECTF{next_level_play}

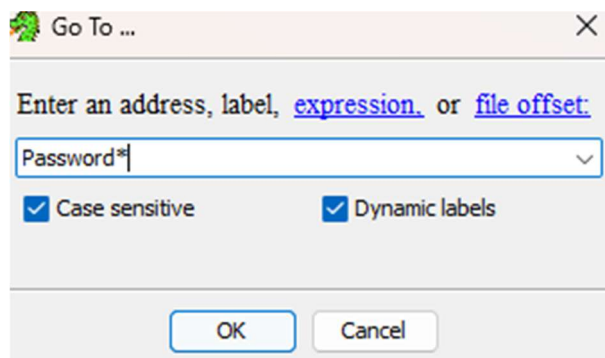
```
Password:      youshallnotpass  
FECTF{next_level_play}Для продолжения нажмите любую клавишу . . . |
```

Разминка. Этап 3

При запуске программа запрашивает пароль при вводе случайного выдает, что он не верен

```
Password: 12314
Wrong, think again
Для продолжения нажмите любую клавишу . . . |
```

По уже отработанной схеме идём в гидру, ищем вхождение строки



Переходим по референсу

				s_Password:_140020498		XREF[1]:	FUN_140015fab:140015fef(*)
140020498	50 61 73	ds	"Password: "				
	73 77 6f						
	72 64 3a ...						
1400204a3	00	??	00h				
1400204a4	00	??	00h				
1400204a5	00	??	00h				
1400204a6	00	??	00h				
1400204a7	00	??	00h				

Видим дизассемблированную функцию

```

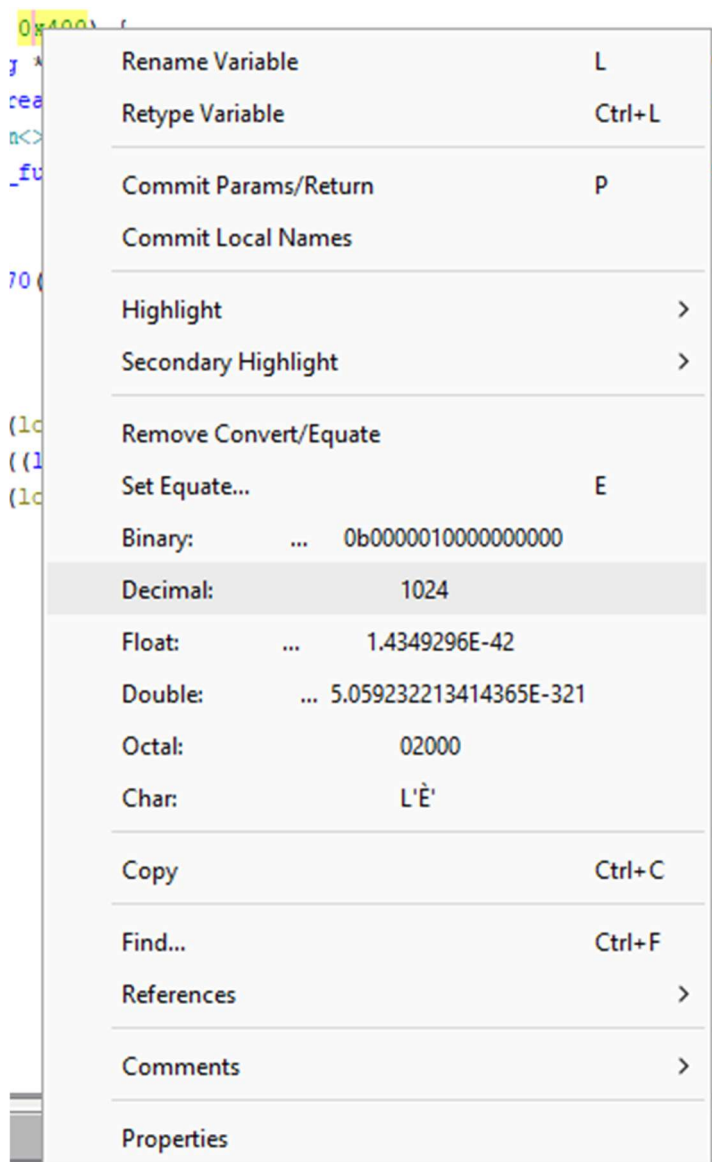
void FUN_140015fa0(void)
{
    longlong *p1Var1;
    basic_ostream<> *this;
    longlong lVar2;
    undefined4 *puVar3;
    undefined1 local_178 [32];
    undefined4 local_158 [2];
    longlong local_150 [7];
    int local_114 [56];
    undefined4 local_34;
    ulonglong local_20;

    puVar3 = local_158;
    for (lVar2 = 0x22; lVar2 != 0; lVar2 = lVar2 + -1) {
        *puVar3 = 0xffffffff;
        puVar3 = puVar3 + 1;
    }
    local_20 = DAT_140024040 ^ (ulonglong)local_158;
    __CheckForDebuggerJustMyCode(&DAT_14002a06b);
    thunk_FUN_140015bd0(local_150);
    local_114[0] = 0;
    thunk_FUN_140011e70((longlong *)cout_exref, "Password: ");
    std::basic_istream<>::operator>>((basic_istream<> *)cin_exref, local_114);
    if (local_114[0] == 0x400) {
        p1Var1 = (longlong *)thunk_FUN_140011e70((longlong *)cout_exref, "\n");
        this = (basic_ostream<> *)thunk_FUN_140011df0(p1Var1, (longlong)local_150);
        std::basic_ostream<>::operator<<
            (this, (_func_basic_ostream<>_ptr_basic_ostream<>_ptr *)&LAB_140011046);
    }
    else {
        thunk_FUN_140011e70((longlong *)cout_exref, "Wrong, think again\n");
    }
    system("pause");
    local_34 = 0;
    thunk_FUN_140013f80(local_150);
    _RTC_CheckStackVars((longlong)local_178, (int *)&DAT_14001ff40);
    thunk_FUN_1400167d0(local_20 ^ (ulonglong)local_158);
    return;
}

```

Запрашиваемый пароль инструкцией cin_exref записывается в local_114

Потом local_114 сравнивается с 0x400 – это и есть проверка пароля, но 0x400 – это шестнадцатеричное представление объекта. Ждем на 0x400 ПКМ, видим контекстное окно. Так как в определениях показано, что local_114 имеет тип int, то это десятичное число



1024 – искомый пароль, вводим его, получаем флаг

FECTF{ready_for_anything}

Password: 1024

FECTF{ready_for_anything}

Для продолжения нажмите любую клавишу . . . |